Ad Soyad: _____	**Tarih:** _____
Lab 10 – Hafta 10 – *İleri Çok Kanallı Programlama: condition_variable, async, thread pool, Qt*	

Lab Hakkında

Bu laboratuvar çalışmasında Bölüm 10'da ele alınan ileri çok kanallı programlama konularını uygulamalı olarak deneyeceksiniz. `std::condition_variable` ile üretici-tüketici kuyruğu kuracak, `std::async` ve `std::future` ile asenkron hesaplama yapacak, basit bir thread havuzu inşa edecek ve son olarak Qt'nin `QtConcurrent` modülü ile UI entegrasyonu gerçekleştireceksiniz.

Süre: 3 saat **Gereksinimler:** g++ (C++20 desteği), `-pthread` bayrağı, Qt 6 (son çalışma için)

Derleme Komutları

Saf C++ örnekleri için:

```
g++ -std=c++20 -pthread -O2 -o program program.cpp
./program
```

Qt örnekleri için CMake kullanacağız (son çalışma).
Lab başlangıç kodları: `out/LabExamples/Lab10/` dizininde mevcut. Her çalışma için boş bir `calismaN.cpp` bulacaksınız; bunları tamamlayarak ilerleyin.

1 Çalışma 1: condition_variable ile Üretici/Tüketici (30 dk)

Ön Bilgi

`std::condition_variable`, bir izleğin başka bir izlekten gelen bir olayı uykuda bekleyebilmesini sağlar. `wait(lock, predicate)` çağrısı mutex'i serbest bırakıp uyur; `notify_one` çağrısıyla uyandırılır. Predicate parametresi sahte uyanmalara karşı korumayı sağlar.

Adım 1: Üretici/Tüketici İskeletinin İncelenmesi

Lab başlangıç dizininden `calisma1.cpp` dosyasını açın:

Listing 1: Üretici/Tüketici İskelet Kodu

```cpp
#include <queue>
#include <mutex>
#include <condition_variable>
#include <thread>
#include <iostream>

std::queue<int> kuyruk;
std::mutex mtx;
std::condition_variable cv;

void uretici() {
    for (int i = 1; i <= 5; ++i) {
        // GOREV 1: kuyruga i degerini ekle (mutex korumali)
        // GOREV 2: bir tuketici uyandir
        std::this_thread::sleep_for(std::chrono::milliseconds(200));
    }
}

```

```
void tuketici() {
    for (int i = 0; i < 5; ++i) {
        std::unique_lock<std::mutex> kilit(mtx);
        // GOREV 3: kuyruk bos olmayana kadar bekle (predicate ile)
        int deger = kuyruk.front();
        kuyruk.pop();
        kilit.unlock();
        std::cout << "Alindi: " << deger << "\n";
    }
}

int main() {
    std::thread t1(uretici);
    std::thread t2(tuketici);
    t1.join();
    t2.join();
    return 0;
}
```

Gorev 1.1: Kuyruğa ekleme (5 dk)

`uretici` fonksiyonundaki "GOREV 1" yorumunu uygun kod ile değiştirin. Mutex'i `std::lock_guard` ile sarmalayın ve `i` değerini `kuyruk.push` ile ekleyin. Not: kilit kapsamını minimum tutun (kilidi `push` sonrası bırakın ki `notify` dışarıdan yapılsın).

Gorev 1.2: notify_one çağrısı (2 dk)

"GOREV 2" yorumunu `cv.notify_one();` ile değiştirin. Bu çağrı mutex kapsamının **dışında** olmalı; neden olduğunu düşünün ve lab defterinize bir cümle yazın.

Gorev 1.3: wait + predicate (3 dk)

`tuketici` fonksiyonundaki "GOREV 3" yorumunu aşağıdaki kod ile değiştirin:

```
cv.wait(kilit, [] { return !kuyruk.empty(); });
```

Programı derleyip çalıştırın. Beklenen çıktı: 1, 2, 3, 4, 5.

Gorev 1.4: Predicate'i kaldır – bozuk versiyonu gör (5 dk)

Predicate parametresini kaldırıp `cv.wait(kilit);` yapın. Programı 5 kez çalıştırın. Her zaman aynı sonucu alıyor musunuz? Bazı çalıştırmalarda program takılabilir veya garip sonuçlar verebilir. Nedeni nedir? (Sahte uyanma + kayıp uyandırma)

Gorev 1.5: Çoklu tüketici (10 dk)

Tüketici sayısını 3'e çıkarın. Üreticinin 15 iş ürettiğinden emin olun. Her tüketici hangi değerleri aldığını yazdırsın. Hangi tüketicinin hangi değerleri aldığını çalıştırmadan önce tahmin edebilir misiniz? Neden?

Kilit Kapsamı İpucu

Kilit altında notify çağırmak verimlilik kaybı yaratır: uyanan izlek hemen tekrar mutex'i beklemeye başlar. Önce kilidi serbest bırakıp sonra `notify_one` çağırmak en iyi pratiktir.

2 Çalışma 2: Paralel Toplam Hesaplama (30 dk)

Ön Bilgi

`std::async(std::launch::async, f, args...)` bir fonksiyonu yeni bir izlekte başlatır ve `std::future<T>` döndürür. `future.get()` ile sonucu alırsınız; sonuç hazır değilse o ana kadar bloklarsınız. İstisnalar future aracılığıyla aktarılır.

Adım 1: Sıralı Toplam – Başlangıç Noktası

`calisma2.cpp` dosyasını açın:

Listing 2: Sıralı Toplam

```
#include <iostream>
#include <numeric>
#include <vector>
#include <chrono>

long long kismiTopla(const std::vector<int>& v,
                     std::size_t baslangic,
                     std::size_t bitis) {
    long long toplam = 0;
    for (std::size_t i = baslangic; i < bitis; ++i) {
        toplam += v[i];
    }
    return toplam;
}

int main() {
    constexpr std::size_t N = 40'000'000;
    std::vector<int> veri(N);
    std::iota(veri.begin(), veri.end(), 1);

    auto t0 = std::chrono::steady_clock::now();
    long long toplam = kismiTopla(veri, 0, N);
    auto t1 = std::chrono::steady_clock::now();

    auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(
        t1 - t0).count();
```

```
    std::cout << "Seri: " << toplam << " (" << ms << " ms)\n";
    return 0;
}
```

Gorev 2.1: Sıralı versiyonu ölç (5 dk)

Programı derleyip çalıştırın. `-O2` bayrağı ile de deneyin:

```
g++ -std=c++20 -pthread -O2 -o calisma2 calisma2.cpp
```

Seri sürenizi kaydedin.

Gorev 2.2: 4 işçi ile async paralel (15 dk)

`main` fonksiyonunu değiştirin: 4 `std::async` çağrısı ile vektörü 4 eşit parçaya bölüp her parçayı ayrı izlekte hesaplatın. Sonuçları `future.get()` ile toplayıp "Paralel: " etiketiyle yazdırın. Zamanı ölçün ve seri ile karşılaştırın.
Şablon:

```
std::vector<std::future<long long>> futurelar;
std::size_t parca = N / 4;
for (int i = 0; i < 4; ++i) {
    // TODO: each chunk -> async(launch::async, kismiTopla,
        cref(veri), bas, son)
}
long long toplam = 0;
for (auto& f : futurelar) toplam += f.get();
```

Gorev 2.3: Hızlanma analizi (10 dk)

4 işçi ile ölçtüğünüz süreyi kullanarak:

Versiyon	Süre (ms)	Hızlanma
Seri (-O0)		1.0x
Seri (-O2)		
Paralel 4 işçi (-O2)		

Hızlanma tam 4x oldu mu? Neden? (Amdahl yasası, vektör kopya maliyeti, cache bozulması)

3 Çalışma 3: Basit Thread Pool İnşası (45 dk)

Ön Bilgi

Thread havuzu, önceden oluşturulmuş sabit sayıda işçi izleği tutar ve gelen görevleri bu işçilere dağıtır. Her görev için yeni izlek yaratmak yerine mevcut işçilere iş verilir. Bu büyük overhead tasarrufu sağlar.

Adım 1: İskelet Kodun İncelenmesi

`calisma3.cpp` dosyasında eksik bir `ThreadPool` sınıfı bulacaksınız:

Listing 3: ThreadPool İskelet

```
#include <thread>
#include <vector>
#include <queue>
#include <mutex>
#include <condition_variable>
#include <future>
#include <functional>

class ThreadPool {
public:
    explicit ThreadPool(std::size_t n);
    ~ThreadPool();

    template<typename F, typename... Args>
    auto submit(F&& f, Args&&... args)
        -> std::future<std::invoke_result_t<F, Args...>>;

private:
    void isciDongusu();

    std::vector<std::thread> isciler_;
    std::queue<std::function<void()>> kuyruk_;
    std::mutex mtx_;
    std::condition_variable cv_;
    bool kapaniyor_ = false;
};
```

Gorev 3.1: Kurucu – işçileri başlat (10 dk)

`ThreadPool::ThreadPool(std::size_t n)` kurucusunu yazın. `n` kadar `std::thread` oluşturup her birini `&ThreadPool::isciDongusu` ile başlatın. `n == 0` ise `n = 1` kullanın.

Gorev 3.2: İşçi döngüsü (10 dk)

`isciDongusu` fonksiyonunu yazın:

1. Sonsuz döngü başlat
2. `unique_lock` al, `cv.wait(kilit, ...)` ile "kuyruk boş değil veya kapanıyor" koşulunu bekle
3. Kapanıyor ve kuyruk boşsa çık
4. Kuyruktan görevi al, kilidi serbest bırak, görevi çalıştır

Gorev 3.3: submit metodu (15 dk)

Sınıfın dışında (ama header aynı dosyada) `submit` şablon fonksiyonunu yazın. İpuçları:

1. `using R = std::invoke_result_t<F, Args...>;`
2. `auto gorev = std::make_shared<std::packaged_task<R()»(...);`
3. `std::bind` ile argümanları ekle
4. `gorev->get_future()` ile future al
5. Kuyruğa `[gorev] { (*gorev)(); }` lambda'yı ekle
6. `cv.notify_one()`
7. Future'ı döndür

Gorev 3.4: Destructor + test (10 dk)

`~ThreadPool` destructor'ında:

1. `kapaniyor_ = true` (kilit altında)
2. `cv.notify_all()`
3. Tüm işçilere `join`

`main` fonksiyonunda:

```
ThreadPool havuz(4);
std::vector<std::future<int>> f;
for (int i = 0; i < 16; ++i)
    f.push_back(havuz.submit([i] { return i * i; }));
long long toplam = 0;
for (auto& x : f) toplam += x.get();
std::cout << "Toplam kareler: " << toplam << "\n";
// Beklenen: 0^2 + 1^2 + ... + 15^2 = 1240
```

4 Çalışma 4: OBYS Toplu GPA Hesaplama (45 dk)

Ön Bilgi

Önceki çalışmada yazdığınız `ThreadPool`'u kullanarak 5000 öğrencinin GPA'sını paralel hesaplayacak, seri ve paralel sürelerini kıyaslayacaksınız.

Adım 1: Veri Modeli

`calisma4.cpp` dosyası OBYS veri modelini ve test verisi üreticiyi içerir:

Listing 4: OBYS Veri Modeli

```
struct DersNotu { int kredi; std::string harf; };
struct Ogrenci {
    int numara;
    std::string ad;
    std::vector<DersNotu> notlar;
```

```
};
struct GPASonuc { int numara; double gpa; };

double harfPuan(const std::string& h);
GPASonuc hesaplaGPA(const Ogrenci& o);
std::vector<Ogrenci> testVerisi(int n);
```

Gorev 4.1: Seri versiyon (10 dk)

5000 öğrenci üretin, hepsini sıralı olarak `hesaplaGPA` ile işleyin, süreyi ölçün. Sonucu bir değişkende tutun.

Gorev 4.2: Thread havuzu ile (15 dk)

Aynı işi `ThreadPool` ile paralel yapın. Her öğrenci için ayrı bir submit çağrısı yapın, future vektörü tutun, sonuçları birleştirin. Süreyi ölçün. Kaç kat hızlandı?

Gorev 4.3: Batch optimizasyonu (15 dk)

Her öğrenci yerine 100 öğrencilik batch'ler halinde submit yapın. Batch içinde tüm GPA'ları tek işçi hesaplasın, sonuçları vektör olarak döndürsün.

```
havuz.submit([&, baslangic, son] {
    std::vector<GPASonuc> yerel;
    for (auto i = baslangic; i < son; ++i)
        yerel.push_back(hesaplaGPA(ogrenciler[i]));
    return yerel;
});
```

Batch ile süre nasıl değişti? Neden?

Gorev 4.4: Karşılaştırma tablosu (5 dk)

Üç yöntemi tabloya yazın:

Yöntem	Süre (ms)	Hızlanma
Seri (1 izlek)		1.0x
Havuz + her öğrenci submit		
Havuz + 100'lü batch		

5 Çalışma 5: Qt ile Responsive UI (30 dk)

Ön Bilgi

`QtConcurrent::mapped` bir kapsayıcının her elemanına bir fonksiyonu paralel uygular. `QFutureWatcher` ile ilerleme sinyalleri alınır. `QProgressDialog` bu sinyalleri doğrudan görsel progress bar'a dönüştürür ve iptal butonu sağlar.

Adım 1: CMake Projesi Hazırlığı

`calisma5/` dizininde CMake iskeleti bulacaksınız:

Listing 5: CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(lab10_calisma5 LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_AUTOMOC ON)
find_package(Qt6 REQUIRED COMPONENTS Core Widgets Concurrent)
add_executable(lab10_calisma5 main.cpp)
target_link_libraries(lab10_calisma5
    PRIVATE Qt6::Core Qt6::Widgets Qt6::Concurrent)
```

Gorev 5.1: main.cpp iskeleti (5 dk)

`main.cpp`'i inceleyin; içinde bir `QWidget` alt-sınıfı var. Bu widget'ta bir buton ve bir liste var. Butona tıklandığında 50 elemanlı bir vektör oluşturulup her eleman için ağır hesaplama başlatılacak.

Gorev 5.2: QtConcurrent::mapped kullanımı (10 dk)

`buton` tıklama slotunu yazın:

1. `QProgressDialog` oluştur
2. `QFutureWatcher<int>` oluştur
3. Watcher sinyallerini dialog'a bağla (progressRangeChanged, progressValueChanged, finished, canceled)
4. `QtConcurrent::mapped(veriler, agirIslem)` ile future al
5. `watcher->setFuture(future)`

Gorev 5.3: Test ve gözlem (15 dk)

Programı derleyin, çalıştırın. Butona basın:

- Progress bar doluyor mu?
- Pencereyi başka yere taşıyabiliyor musunuz (UI donmuyor mu)?
- İptal butonuna bastığınızda durur mu?
- İşçi sayısı kaç? (`QThread::idealThreadCount()` değerine bakın)

Bonus: QtConcurrent::mapped ile Not Dönüşümü (Opsiyonel)

Bonus

Aşağıdaki görev zorunlu değildir; bitirenler ekstra puan alır.

Bonus: Harf notlarını puana dönüştürme

Çalışma 5'in `agirIslem` fonksiyonu yerine aşağıdaki dönüşümü yapın: `QList<QString> harfler = "AA", "BA", ...`'den `QList<double> puanlar`'a `QtConcurrent::mapped` ile paralel dönüşüm. Sonucu bir `QListWidget`'ta gösterin.

Teslim Bilgileri

Teslim

- Tüm `.cpp` dosyalarını ve ekran görüntülerini sıkıştırıp sisteme yükleyin.
- Dosya adı formatı: `Lab10_OgrenciNo_AdSoyad.zip`
- Çalışma 5 için `build/` dizinini değil yalnızca kaynakları gönderin.
- Teslim tarihi: Lab saatinden itibaren 1 hafta.

Çalışma	Puan
Çalışma 1: condition_variable ile Üretici/Tüketici	15
Çalışma 2: Paralel Toplam (async)	15
Çalışma 3: Thread Pool İnşası	30
Çalışma 4: OBYS Toplu GPA Hesaplama	25
Çalışma 5: Qt Responsive UI	15
Bonus: QtConcurrent::mapped	+10
Toplam	**100 (+10)**

Başarılar!